

ICDE 2025 论文解读：基于时间戳的在线数据库事务隔离级别检查

标题：Online Timestamp-based Transactional Isolation Checking of Database Systems
(基于时间戳的在线数据库事务隔离级别检查)

作者：李和煦，魏恒峰，欧阳鸿荣，陈育兴，杨娜，张若皓，潘安群

关键词：隔离级别，可串行化，快照隔离，在线隔离检查

论文链接：<https://arxiv.org/abs/2504.01477>

代码链接：<https://github.com/FertileFragrance/TimeKiller>

摘要：

可串行化（SER）和快照隔离（SI）是数据库系统中广泛使用的事务隔离级别，隔离检查问题用于判断数据库事务的给定执行历史记录是否满足指定的隔离级别。然而，现有的 SER 和 SI 检查器，无论是传统的黑盒检查器还是最新的基于时间戳的白盒检查器，都是离线检查，并且需要完整的执行历史才能构建依赖关系图，因此它们不适用于连续且不断增长的历史。

本文通过将基于时间戳的隔离检查方法扩展至在线环境以解决在线隔离检查问题。具体而言，我们设计了一个高效的基于时间戳的离线 SI 检查器 Chronos，它是增量式的，无需为整个历史记录构建依赖关系图，因此非常适合在线场景。我们进一步将 Chronos 扩展为在线 SI 检查器 Aion，以解决在线环境特有的关键挑战。此外，我们还开发了 Aion-SER 用于在线 SER 检查。实验表明，Chronos 可在几秒钟内检查完多达 100 万个事务的离线历史，性能远超现有的 SI 检查器。此外，Aion 和 Aion-SER 的吞吐量约为每秒 1.2 万个事务，证明了它们在在线隔离检查中的实用性。

引言

数据库事务提供了一种“全有或全无”的机制，隔离在共享数据上的并发计算，大大简化了客户端编程。可串行化（SER）是事务隔离的黄金标准，它确保事务看起来是顺序执行的，PostgreSQL、CockroachDB、YugabyteDB 等数据库提供了 SER 的支持；为了更好的性能，YugabyteDB、Dgraph、MongoDB、TiDB 等数据库提供一种稍弱的隔离级别，快照隔离（SI）。

正确实现数据库是十分困难的，许多数据库未能像声称的那样提供 SER 或 SI，给定执行历史，判断其是否满足某个隔离级别对于数据库测试来说非常重要。现有的检查器可分为

黑盒检查器和白盒检查器，黑盒检查器只从外部观察中提取事务信息，而白盒检查器会利用数据库系统的内部信息（如时间戳），它们具有以下局限性：

1. 对于黑盒检查器（例如 dbcop、Cobra、PolySI 和 Viper）来说，由于黑盒检查是 NP 完全问题，这些检查器具有高计算复杂性，无法扩展到包含数万个事务的大历史；
2. 对于白盒检查器（例如 Emme）来说，它能够在多项式时间内解决问题，但仍需花费较多计算资源来构建依赖图，效率不高；
3. 所有的检查器除了 Cobra 外都不支持在线检查，而 Cobra 只支持在线 SER 检查不支持 SI 检查，且它需要周期性地手动将“栅栏事务”注入到工作负载中，这在生产环境往往是不可接受的。

我们通过将基于时间戳的隔离检查方法扩展到在线环境，解决了数据库系统的在线事务隔离检查问题。一方面，在线检查器必须高效，以跟上数据库吞吐量，这会生成一个持续增长的历史；另一方面，在线检查器不能假设事务是按其开始或提交时间戳的升序收集的，因为数据库系统中的异步性使这一点不可行。这引入了在线检查器独有的三个关键挑战，我们在工作中解决了这些挑战。

1. 由于异步性，我们不能在收集一个事务后立即断定它的正确性；
2. 新收集的事务可能具有一个较小的开始时间戳，它会影响那些具有更大（开始或提交）时间戳的已经检查过的事务，这些事务需要重新检查；
3. 为了保证长期的可用性和无限制的内存使用，检查器必须支持垃圾回收。

预备知识：SI 的语义

SI 最初的定义是基于操作定义的，描述了应当如何实现 SI，如下图所示。

Algorithm 1 Operational Semantics of Snapshot Isolation

log: the log of committed transactions

```
1: procedure START( $T$ )
2:    $T.start\_ts \leftarrow \text{REQUEST}(\mathcal{O})$ 
3:   return ok
4: procedure WRITE( $T, k, v$ )
5:    $T.buffer \leftarrow T.buffer \circ \langle k, v \rangle$ 
6:   return ok
7: procedure READ( $T, k$ )
8:   return value of  $k$  from  $T.buffer$  and log as of
    $T.start\_ts$ 
9: procedure COMMIT( $T$ )
10:   $T.commit\_ts \leftarrow \text{REQUEST}(\mathcal{O})$ 
11:  if  $T$  conflicts with some concurrent transaction
12:    return aborted
13:   $log \leftarrow log \circ \langle T.buffer, T.commit\_ts \rangle$ 
14:  return committed
```

在事务开始时，会请求 time oracle 分配一个开始时间戳；当事务进行写操作时，会将键值对暂时写到事务自己的缓冲区里；当事务进行读操作时，如果自己的缓冲区里有相应的值则直接读取，否则从全局的日志中读相应的值，能看到日志的哪些内容是由其开始时间戳决定的；事务提交时，同样请求 time oracle 分配一个提交时间戳，然后检查其是否与其他并发事务有写写冲突，没有则将其缓冲区的内容写入日志，成功提交。

由此可见，SI 的原始定义（操作语义）中就有时间戳，许多数据库也是这样实现 SI 的。为了方便地使用时间戳来检查 SI，我们采用 SI 的公理语义（而非现有工具采用的依赖图相关的定义），此处省略相关公理的形式化定义，直观地说，满足 SI 等价于满足以下 4 个公理：

- INT（内部一致性公理）：一个事务内部对同一个键可能进行了多次操作（读或写），在进行第二次或以上的操作时，如果这次是读操作，那么读的结果必须与上一次在这个键上操作的结果相同。例如一个事务第一次在 x 上读到了 3，第二次又在 x 上进行了读操作，应该仍然读到 3；或者一个事务在 y 上写了 5，然后读 y 应该能够读到自己写的 5，直到自己又对 y 写了其他的值。
- EXT（外部一致性公理）：一个事务在某个键上首次进行了操作并且是读操作，那么它应该读到其他对于其来说可见的已提交的事务在这个键上写的值，并且应该读到最新的值。可见性由时间戳决定，即一个事务的开始时间戳大于等于另一个事务的提交时间戳，则它能看到另一个事务。值的“新旧”由提交时间戳决定，在同一个键上，后提交的事务写的值更“新”。
- PREFIX（前缀公理）：在使用时间戳进行检查时该公理天然满足，不详细介绍。

- NOCONFLICT (无冲突公理)：两个并发的事务不能在同一个键上进行写操作。并发的事务是指两个事务的时间戳有重叠，而不是一个事务的提交时间戳小于等于另一个事务的开始时间戳。该公理本质上相当于操作语义中事务提交时的并发检查。

此外，数据库在实现过程中一般都会假定满足一个额外的 SESSION 公理，即同一个会话中后开始的事务一定要在前一个事务提交之后进行，即后一个事务的开始时间戳需要大于前一个事务的提交时间戳。满足以上 4 个公理和 SESSION 公理的隔离级别被称作强会话快照隔离 (Strong Session SI)，它是一个比 SI 略强的变体，本文中检查 SI 的同时也会检查 SESSION 公理。

离线和在线检查算法

Algorithm 2 CHRONOS: the offline SI checking algorithm

$last_sno \in SID \rightarrow \mathbb{N}$: sno of the last transaction already processed in each session, initially -1
 $last_cts \in SID \rightarrow TS$: $commit_ts$ of the last transaction already processed in each session, initially $\perp_{ts}$
 $frontier \in K \rightarrow V$: the last committed value of each key, initially $\perp_v$
 $ongoing \in K \rightarrow 2^{TID}$: the set of ongoing transactions on each key, initially $\emptyset$
 $int_val \in TID \rightarrow (K \rightarrow V)$: the last read/written value of each key in each transaction, initially $\perp_v$
 $ext_val \in TID \rightarrow (K \rightarrow V)$: the last written value of each key in each transaction, initially $\perp_v$

```

1: procedure CHECKSI( $\mathcal{H}$ )           ▷  $\mathcal{H}$  contains the initial transaction  $\perp_T$ 
2:   sort TS in ascending order
3:   for  $ts \in TS$ 
4:      $T \leftarrow$  the transaction that owns the timestamp  $ts$ 
5:      $sid \leftarrow T.sid$     $tid \leftarrow T.tid$     $T.wkey \leftarrow \emptyset$ 
6:     if  $ts = T.start\_ts$            ▷ start event of  $T$ 
7:       if  $T.sno \neq last\_sno[sid] + 1 \vee T.start\_ts < last\_cts[sid]$ 
8:         report a violation of SESSION
9:          $last\_sno[sid] \leftarrow T.sno$ 
10:         $last\_cts[sid] \leftarrow T.commit\_ts$ 
11:        for ( $op = \_ (k, v)$ )  $\in T.ops$            ▷ in program order
12:          if  $op = R(k, v)$ 
13:            if  $int\_val[tid][k] = \perp_v$            ▷ external read
14:              if  $v \neq frontier[k]$ 
15:                report a violation of EXT
16:            else if  $int\_val[tid][k] \neq v$            ▷ internal read
17:              report a violation of INT
18:          else                               ▷  $op = W(k, v)$ 
19:             $T.wkey \leftarrow T.wkey \cup \{k\}$ 
20:             $ext\_val[tid][k] \leftarrow v$ 
21:             $ongoing[k] \leftarrow ongoing[k] \cup \{tid\}$ 
22:             $int\_val[tid][k] \leftarrow v$ 
23:        else                               ▷ commit event of  $T$ 
24:          if  $T.start\_ts > T.commit\_ts$            ▷ check Eq. (1)
25:            report an error
26:          for  $k \in T.wkey$ 
27:             $ongoing[k] \leftarrow ongoing[k] \setminus \{tid\}$            ▷ except  $T$  itself
28:            if  $ongoing[k] \neq \emptyset$ 
29:              report a violation of NOCONFLICT
30:             $frontier[k] \leftarrow ext\_val[tid][k]$ 
31:             $int\_val[tid] \leftarrow \emptyset$            ▷ gc  $int\_val$ 
32:             $ext\_val[tid] \leftarrow \emptyset$            ▷ gc  $ext\_val$ 
33:             $T \leftarrow T \setminus \{T\}$            ▷ gc  $T$ 

```

上图展示了 Chronos 的离线检查算法，它接受事务历史作为参数，检查其是否满足 SI。历史中的每个事务包含以下信息：

- T.tid: 事务的唯一 ID，我们用 TID 表示历史中所有事务 ID 组成的集合；
- T.sid: 事务所属的会话的唯一 ID，我们用 SID 表示历史中所有会话 ID 组成的集合；
- T.sno: 事务在其会话中的自增序列号；
- T.ops: 事务的读写操作列表，每个操作包含操作类型（读或写）、键和值；
- T.start_ts 和 T.commit_ts: 事务的开始时间戳和提交时间戳，我们用 TS 表示历史中所有（开始和提交）时间戳组成的集合。

Chronos 模拟数据库的执行，假设历史中事务的开始和提交事件按其时间戳的顺序执行，同时实时检查相应的公理。为此，Chronos 首先把所有时间戳按照升序排序（第 2 行），然后遍历这些时间戳，逐一处理每个事务的开始事件和提交事件（第 3 行）。在这一过程中，Chronos 维护了两个映射：frontier 将每个键映射到当前最后一个提交的值，ongoing 将每个键映射到正在进行的写这个键的事务集。

当前处理的时间戳如果是一个开始事件，Chronos 会检查 SESSION、INT 和 EXT 公理（第 6 行），如果是一个提交事件则检查 NOCONFLICT 公理（第 23 行）。

检查 SESSION 公理（第 7-10 行）：Chronos 使用 last_sno 和 last_cts 来维护每个会话中已处理的最后一个事务的 sno 和 commit_ts，它检查当前事务是否紧接着其会话中的上一个事务之后且开始时间戳是否大于上一个事务的提交时间戳。

检查 INT 和 EXT 公理（第 12-17 行）：Chronos 使用 int_val 跟踪当前事务在某个键上最后读取或写入的值（第 22 行）。检查每个读操作时，如果读到了 int_val 记录的非初始值（即这个键之前被读取或写入过），那么这是一个内部读（第 16 行），此时检查是否满足 INT 公理；如果是 int_val 的初始值，则这是一个外部读（第 13 行），EXT 公理要求它应该读到这个键上的最后提交的值，据此检查是否满足 EXT 公理（第 14 行）。

对于一个键 k，Chronos 每当写入 k 的事务提交时更新 frontier[k]（第 30 行），ext_val 维护了一个事务在某个键上最后写入的值，因此当它提交时 frontier 中就会更新到最新的值。

检查 NOCONFLICT 公理（第 27-29 行）：当处理提交事件的时间戳时（第 23 行），对于当前事务每一个写入的键（在第 19 行被记录下来），Chronos 通过检查 ongoing[k]是否为空（在第 21 行被记录下来）来判断当前事务是否与其他正在执行的事务冲突（第 27 行）。

在在线环境中，事务历史并不事先已知，在线检查算法 Aion 会逐个接收事务并增量式地检查 SI。我们假定事务在接收时会保留会话顺序。

由于异步性，Aion 无法预见事务会按其开始和提交时间戳的升序收集，这会给 Aion 带来 3 个挑战：

1. 由于异步性，确定事务是否满足或违反 EXT 公理变得不稳定，这意味着我们不能在事务被收集时立即断言；
2. 新收集的事务可能带有更小的开始时间戳，这会导致在该事务提交后的事务需要重新检查，上面的数据结构 frontier 和 ongoing 需要进行扩展；
3. 为了防止无限的内存使用并促进长时间检查，Aion 必须回收不再需要的数据结构和事务。

Algorithm 3 AION: the online SI checking algorithm

last_sno, last_cts, int_val, and ext_val are the same as in CHRONOS

```

1: procedure ONLINECHECKSI(T) ▷ upon receiving a new transaction T
2:   T.EXT ← ⊥ ▷ T.EXT: whether T satisfies EXT, initially true
3:   set a timer for re-checking EXT for T
4:   if T.start_ts > T.commit_ts ▷ check Eq. (1)
5:     report an error
6:   sid ← T.sid    tid ← T.tid    T.wkey ← ∅
7:   ▷ ① check SESSION, INT, and EXT for T, similarly to CHRONOS
8:   if T.sno ≠ last_sno[sid] + 1 ∨ T.start_ts < last_cts[sid]
9:     report a violation of SESSION
10:  last_sno[sid] ← T.sno
11:  last_cts[sid] ← T.commit_ts
12:  for (op =  $\_ (k, v)$ ) ∈ T.ops
13:    if op = R(k, v)
14:      if int_val[tid][k] = ⊥v ▷ external read
15:        if v ≠ frontier_ts[T.start_ts][k] ⚡
16:          T.EXT ← ⊥
17:        else if int_val[tid][k] ≠ v ▷ internal read
18:          report a violation of INT
19:        else ▷ op = W(k, v)
20:          T.wkey ← T.wkey ∪ {k}
21:          ext_val[tid][k] ← v
22:          int_val[tid][k] ← v
23:          cts ← T.commit_ts
24:          frontier_ts[cts] ← frontier_ts[cts] ⚡
25:          frontier_ts[cts][k] ← ext_val[tid][k] for k ∈ T.wkey
26:          int_val[tid][k] ← ⊥v for k ∈ DOM(int_val[tid]) ▷ reset
27:        ▷ ② re-check NOCONFLICT for transactions overlapping with T
28:        for ts ∈ TS such that T.start_ts ≤ ts ≤ T.commit_ts ⚡
29:          T' ← the transaction that owns the timestamp ts
30:          if ts = T'.start_ts
31:            for k ∈ T'.wkey
32:              ongoing_ts[ts][k] ← ongoing_ts[ts][k] ∪ {T'.tid} ⚡
33:            else ▷ ts = T'.commit_ts
34:              for k ∈ T'.wkey
35:                ongoing_ts[ts][k] ← ongoing_ts[ts][k] \ {T'.tid} ⚡
36:                if ongoing_ts[ts][k] ≠ ∅
37:                  report a violation of NOCONFLICT
38:          ▷ ③ re-check EXT for transactions starting after T commits
39:          keys ← T.wkey
40:          for ts ∈ TS such that ts > T.commit_ts
41:            T' ← the transaction that owns the timestamp ts ⚡
42:            if ts = T'.start_ts
43:              if TIMEOUT(T') has been called
44:                continue
45:              for (op =  $\_ (k, v)$ ) ∈ T'.ops
46:                if op = R(k, v) ∧ int_val[T'.tid][k] = ⊥v ∧ k ∈ keys
47:                  if ext_val[tid][k] ≠ v
48:                    T'.EXT ← ⊥
49:                  else
50:                    T'.EXT ← ⊥
51:                  if k ∈ keys
52:                    int_val[T'.tid][k] ← ⊥v ▷ reset int_val of T'
53:                    keys ← keys \ T'.wkey
54:                    if keys = ∅
55:                      break
56:                  for k ∈ keys
57:                    frontier_ts[ts][k] ← ext_val[tid][k]
58:                    ext_val[tid] ← ∅ ▷ gc ext_val of T
59: procedure TIMEOUT(T) ▷ runs when timeout for T expires
60:   if T.EXT = ⊥
61:     report a violation of EXT
62:   ▷ gc frontier_ts, ongoing_ts, and transactions below timestamp ts
63:   procedure GARBAGECOLLECT(ts) ▷ runs periodically
64:     for ts' ≤ ts
65:       frontier_ts[ts'] ← ∅ ⚡
66:       ongoing_ts[ts'] ← ∅ ⚡
67:       T ← T \ {T ∈ T | T.commit_ts ≤ ts} ⚡

```

上图介绍了 Aion 的在线检查算法。Aion 扩展将 Chronos 使用的 frontier 和 ongoing 扩展为了 frontier_ts 和 ongoing_ts，通过时间戳进行版本控制，frontier[ts] 和 ongoing[ts] 分别表示在时间戳 ts 之前的 frontier_ts 和 ongoing_ts 的最新版本。

当收集到一个新事务时，Aion 会进行 3 步处理：首先 Aion 像 Chronos 一样检查 SESSION、INT 和 EXT 公理（第 6-25 行），然后对那些与该新事务时间戳有重叠的事务检查 NOCONFLICT 公理（第 26-35 行），最后对于在其提交之后的事务重新检查 EXT 公理（第 37-57 行）。

第 1 步（第 6-25 行）：总体与 Chronos 类似，有两点不同。第一点是在检查 EXT 公理时，Aion 需要根据开始时间戳获取在这之前的最新版本的 `frontier_ts`（第 14 行），并且在更新 `frontier_ts` 时也要更新在自己对应的版本上（第 24 行）；第二点是如果发现了 EXT 违反，则暂时不报告（因为这可能是一个 `false alarm` 等待被纠正），而是为该事务设置一个定时器（第 15 行），时间到了才最终认定这是一个 EXT 违反（第 59 行）。

第 2 步（第 26-35 行）：Aion 升序遍历从新收集事务的开始时间戳和提交时间戳之间的所有时间戳来获取那些与之有重叠的事务（第 26 行），然后与 Chronos 类似，在事务开始时根据其写了哪些键将其添加到正在执行的事务中（第 30 行），在事务提交时将其从中删去并检查是否为空（第 33-34 行），唯一的区别在于要根据时间戳进行版本控制。

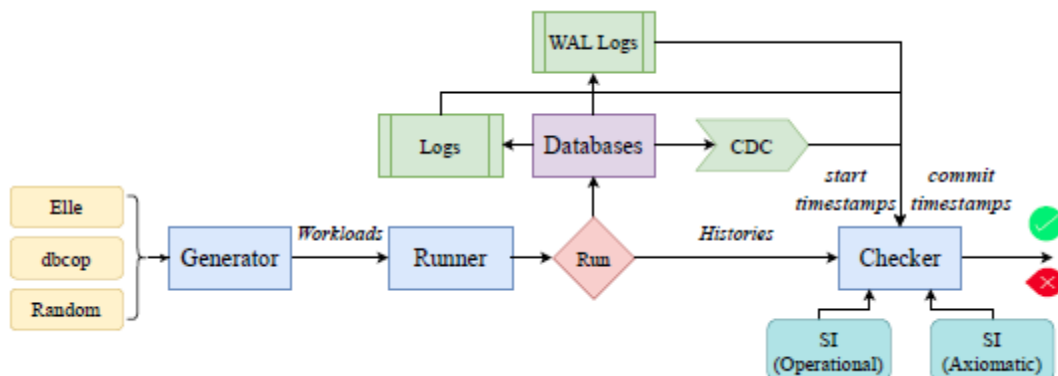
第 3 步（第 37-57 行）：为了对在新事务提交后的事务进行重新检查，Aion 升序遍历比新事务提交时间戳更大的时间戳（第 37 行），如果这是一个开始事件且对应的事务已经超时了，说明它的 EXT 违反结果已经被最终决定不能改变，因此跳过对它的重新检查（第 40-41 行）。当一个事务要被重新检查时，Aion 通过 `ext_val` 来检查 EXT 公理（第 44 行）而非第 1 步中使用的 `frontier_ts`，并且在提交时使用 `ext_val` 更新 `frontier_ts`（第 57 行）。此外，Aion 还做了一个优化（划线部分），它只检查新收集事务写入的且还未被后提交事务覆盖的键，一旦这些键全部被覆盖则跳出重新检查过程（第 55 行）。

以上的检查算法解决了 3 个挑战中的前两个，对于第 3 个挑战，Aion 使用了垃圾回收机制来处理未来大概率不会被用到的数据结构（第 63-66 行）。Aion 周期性地将时间戳小于给定值的 `frontier_ts` 和 `ongoing_ts` 版本以及事务从内存加载到磁盘，万一这些信息会再次被用到，则到磁盘中加载。该垃圾回收机制不影响检查的正确性。

实验效果

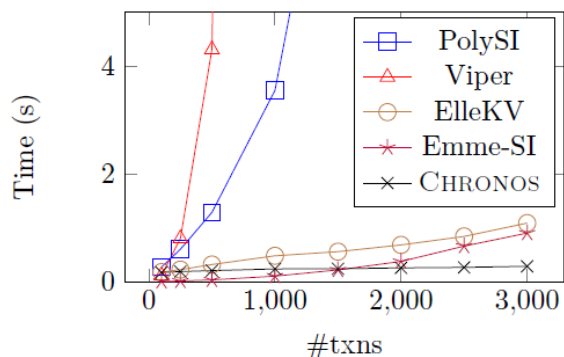
下图展示了基于时间戳的事务隔离级别检查的完整工作流。首先事务生成器会根据一定的参数和策略生成待执行的事务，然后执行器会连接到已部署的数据库执行这些事务形成执行历史。执行结束后数据库中就会保存有事务的信息，其中包括事务的开始时间戳和提交时间戳，这些信息往往保存在数据库的日志、预写日志或变更数据捕获机制中。提取到这

些信息后，检查器（即 Chronos 和 Aion）就可以检查执行历史是否满足某个指定的隔离级别。

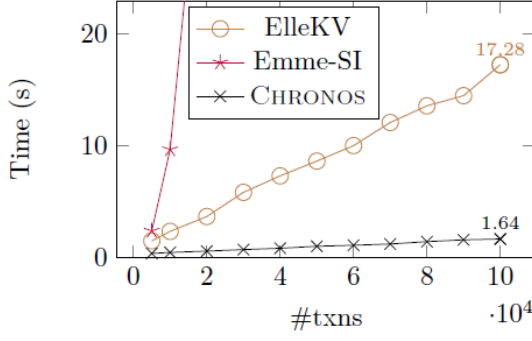


在离线检查的工作流中，以上的每一步需要等待前一步完全结束才能进行。而对于在线检查的工作流，在事务执行的过程中，可以一边收集历史一边将历史发送给检查器，检查器会进行增量式的检查。

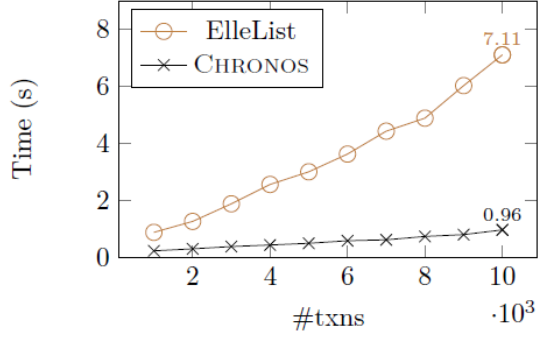
本文采用多种工作负载，在 TiDB、YugabyteDB 和 Dgraph 上对 Chronos 和 Aion 进行了充分的评估，包括性能对比、消融实验、资源开销等。



上图展示了 Chronos 与其他检查器的效率对比，PolySI 和 Viper 两个黑盒工具的检查时间随着事务数量的提升急剧增加，其他两个工具在规模较小时则增长较慢，Emme-SI 在 2000 以上的事务规模时检查时间的增长速度已经开始加快。

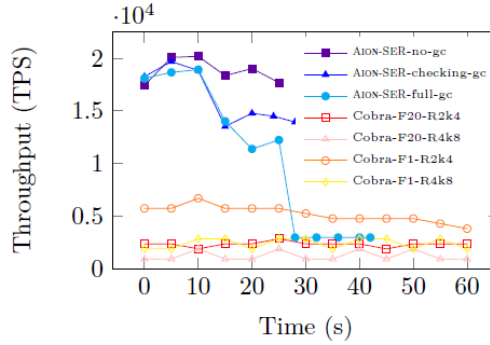


(a) On key-value histories

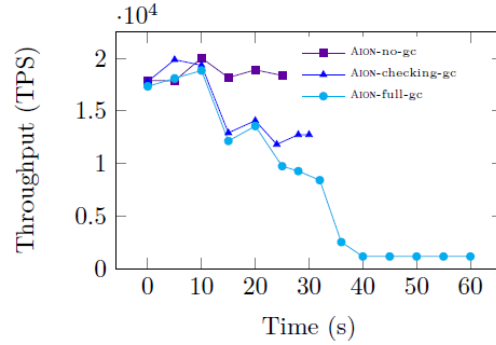


(b) On list histories

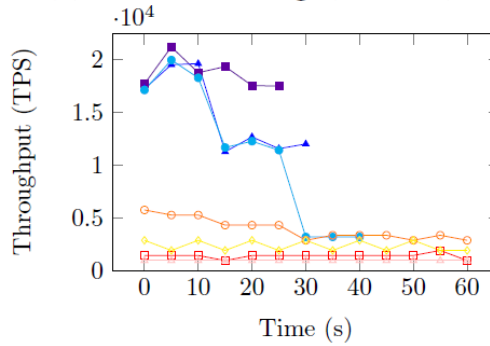
为了探究在事务规模较大时两个工具和 Chronos 的性能对比，我们增大了事务规模，同时为了让 Elle 发挥出优势，也进行了 Chronos 和 Elle 在列表数据结构上的事务历史检查（即 ElleList）的对比。当事务规模较大且快速增加时，Emme-SI 的检查时间也急速增加，相比之下，无论是何种数据结构，Elle 的检查时间看起来是随事务数量线性增加，但仍然比 Chronos 增加得快。



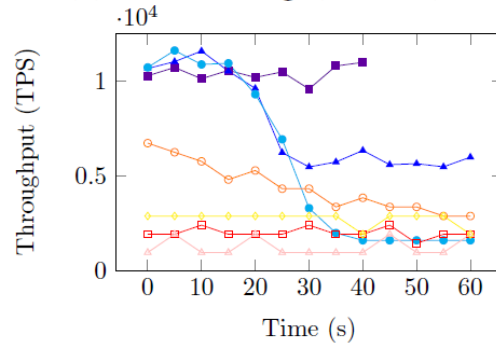
(a) SER checking (Default)



(b) SI checking (Default)



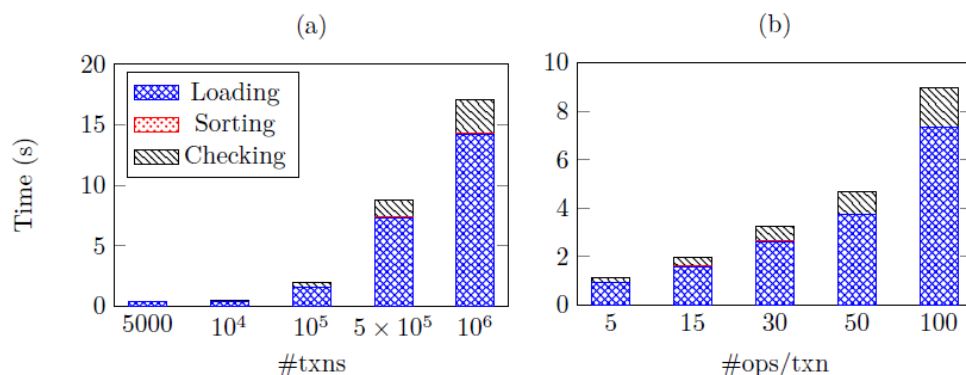
(c) SER checking (RUBiS)



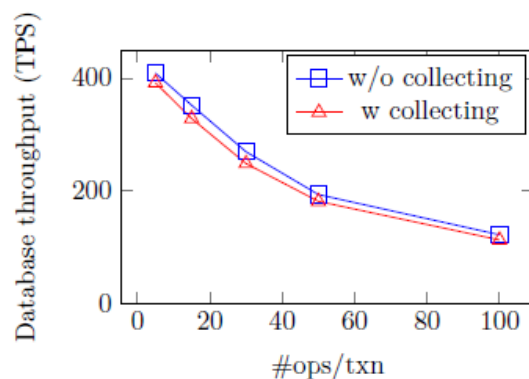
(d) SER checking (Twitter)

上图展示了 Aion 在不同垃圾回收策略下与 Cobra 的性能对比结果。Aion 在完全不进行垃圾回收时最多每秒可检查约 1.8 万个事务；在周期性地一边检查一边垃圾回收时，每秒可

检查约 1.2 万个事务；当设定的内存满后，每秒可检查约 3000 个事务。在绝大多数设定下，Aion 的吞吐量都远高于 Cobra。



上图展示了 Chronos 随着事务规模或每个事务的操作数量增加时各阶段用时，可以看到加载阶段（即把历史从磁盘加载到程序中）总是占据总耗时的大多数，检查阶段只是占了一小部分，说明程序的瓶颈并不在检查算法，而是在于读取磁盘上的文件。



运行 Aion 需要一边执行数据库事务一边将提交的事务收集起来发送给 Aion 检查，上图展示了是否收集事务对于数据库吞吐量的影响，结果显示收集事务会导致数据库的吞吐量降低约 5%，这在实际运行中是可以接受的。

总结

本文通过将基于时间戳的离线隔离检查方法扩展到在线环境，解决了数据库系统中的在线事务隔离检查问题。具体而言，我们提出了高效的离线 SI 检查算法 Chronos，该算法本质上是增量的，我们将它扩展到在线环境，引入了 Aion 算法。我们还开发了能够在线检查 SER 的检查器 Aion-SER。实验结果表明，Chronos 和 Aion-SER 的性能远超现有的检查器，Aion 和 Aion-SER 可以处理在线事务工作负载，持续吞吐量约为每秒 1.2 万个事务，且对数据库的吞吐量影响很小。